Compiladores

Trabalho Prático 1

Analisador léxico e sintático para um subconjunto de Pascal

Nome do Aluno 1 (up20XXXXXXX)
Nome do Aluno 2 (up20XXXXXXX)

Faculdade de Ciências da Universidade do Porto
Outubro 2026

Índice

1 Introdução **3**

2 Ferramentas **4**

2.1 Requisitos 4

2.2 Compilação 4

2.3 Execução 4

2.4 Estrutura de diretórios 4

3 Frontend **5**

3.1 Análise lexical e sintática 5

3.1.1 Tokens 5

3.1.2 Gramática 5

3.1.3 Precedência de operadores 6

3.2 Árvore sintática abstrata (AST) 6

3.3 Erros 7

4 Conclusão **8**

5 Referências **9**

1 Introdução

Este trabalho consiste num analisador léxico e sintático para um pequeno subconjunto da linguagem Pascal. O subconjunto inclui a declaração de variáveis inteiras e booleanas, atribuições, condições (if-then-else), ciclos (while-do), blocos begin...end, operações de leitura e escrita e expressões aritméticas, relacionais e booleanas.

O analisador lexical (Flex) divide o programa em tokens e o analisador sintático (Bison) verifica se esses tokens respeitam a gramática. Durante a análise sintática é construída a árvore sintática abstrata (AST) do programa, que no fim é impressa no ecrã. Se o programa tiver um erro, é indicada a linha onde o erro foi encontrado. O código foi escrito em C.

Nota: o exemplo usado neste documento é o ficheiro `tests/exemplo3.pas`.

2 Ferramentas

2.1 Requisitos

Para compilar e correr o trabalho é preciso ter instalado o `make`, o `flex`, o `bison` (versão 3) e o `gcc`.

2.2 Compilação

A forma mais simples de compilar é usar o Makefile:

```
make
```

Este comando gera os ficheiros `parser.tab.c`, `parser.tab.h` e `lex.yy.c` e cria o executável `pascal`. Também é possível compilar manualmente:

```
bison -d parser.y
flex scanner.l
gcc -o pascal parser.tab.c lex.yy.c
```

Para apagar os ficheiros gerados usa-se `make clean`.

2.3 Execução

O programa recebe como argumento o ficheiro Pascal a analisar e imprime a AST:

```
./pascal caminho/para/ficheiro.pas
```

Para correr todos os exemplos da pasta `tests/` de uma vez:

```
make test
```

2.4 Estrutura de diretórios

```
|-- COMP_TP1/
    |-- scanner.l
    |-- parser.y
    |-- Makefile
    |-- README.pdf
    |-- tests/
        |-- exemplo1.pas
        |-- exemplo2.pas
        |-- exemplo3.pas
        |-- erro.pas
```

`scanner.l` contém o analisador lexical, `parser.y` contém a gramática, a construção e a impressão da AST e a função `main`. Os ficheiros `exemplo*.pas` são programas corretos e o `erro.pas` tem um erro de sintaxe.

3 Frontend

3.1 Análise lexical e sintática

3.1.1 Tokens

O analisador lexical reconhece:

- Palavras reservadas: `program var integer boolean begin end if then else while do write writeln read readln true false div mod and or not`;
- Operadores e símbolos: `:= <> <= >= + - * / = < > ( ) ; : , .`;
- Números inteiros, strings entre plicas (`'texto'`) e identificadores;
- Comentários `{ ... }` e `//`, que são ignorados, tal como os espaços.

As palavras reservadas estão escritas antes da regra dos identificadores. Assim, quando há empate (por exemplo `if`), ganha a primeira regra (*first match*). Nos operadores de dois caracteres, como `<=`, ganha a regra que lê mais caracteres (*longest match*). Um caracter que não pertença à linguagem dá erro.

3.1.2 Gramática

A gramática deste subconjunto é a seguinte:

- Programa: `program {nome}; {declarações} begin {comandos} end.`
- Declarações: opcionais; começam por `var` e têm uma ou mais linhas `{ids} : {tipo};`, com tipo `integer` ou `boolean`
- Comandos: zero ou mais, separados por `;`
- Comandos:
  - Atribuição: `{id} := {expressão}`
  - If-then: `if {expressão} then {comando}`
  - If-then-else: `if {expressão} then {comando} else {comando}`
  - Ciclo while: `while {expressão} do {comando}`
  - Bloco: `begin {comandos} end`
  - Escrita: `write({args})`, `writeln({args})` ou `writeln`, onde os argumentos são expressões ou strings
  - Leitura: `read({ids})` ou `readln({ids})`
- Expressões:
  - Números inteiros, variáveis, `true` e `false`
  - Menos unário (`-`) e negação (`not`)

- Operações aritméticas: `+ - * / div mod`
- Operações booleanas: `and or`
- Operações relacionais: `= <> < <= > >=`
- `(expressão)`

3.1.3 Precedência de operadores

A regra das expressões é ambígua (por exemplo, `2+3*4` tem duas árvores possíveis). Para o Bison escolher a árvore correta, a precedência e a associatividade dos operadores são declaradas com `%left` e `%nonassoc`. Cada linha tem mais prioridade do que a anterior. Seguindo o Pascal, da precedência mais alta para a mais baixa:

- Menos unário e `not` (estão na regra `fator`, por isso aplicam-se só ao operando que vem a seguir)
- Multiplicativos: `* / div mod and` (associativos à esquerda)
- Aditivos: `+ - or` (associativos à esquerda)
- Relacionais: `= <> < <= > >=` (não associativos: `a < b < c` é erro)

O mesmo mecanismo resolve o *dangling else*: o `ELSE` tem mais precedência do que o `THEN`, por isso o `else` fica sempre ligado ao `if` mais próximo. Com estas declarações, o Bison não reporta conflitos.

3.2 Árvore sintática abstrata (AST)

A AST usa um único tipo de nó:

```
typedef struct No { char *nome; struct No *a, *b, *c, *prox; } No;
```

Cada nó tem uma etiqueta (`nome`), até três filhos (`a`, `b`, `c`) e um ponteiro `prox` que liga nós da mesma lista (comandos, variáveis e argumentos). As ações da gramática criam os nós com três funções:

- `no(nome, a, b, c)`: cria um nó;
- `liga(lista, n)`: acrescenta um nó ao fim de uma lista;
- `mostra(n, nivel)`: imprime a árvore, com 2 espaços de indentação por nível.

As etiquetas usadas são `Program`, `Declaracoes`, `Decl`, `Var`, `Corpo`, `Assign`, `If`, `While`, `Bloco`, `Write`, `Writeln`, `Read`, `Readln`, `Int`, `String`, `true`, `false` e o símbolo de cada operador. Um `If` tem 2 filhos (condição e then) ou 3 (com else). O menos unário aparece como `-` com um só filho.

Para o programa `tests/exemplo3.pas`:

```
// ciclo while: soma de 1 a n
program Soma;
var n, i, s : integer;
begin
  readln(n);
  i := 1;
  s := 0;
  while i <= n do
  begin
    s := s + i;
    i := i + 1
  end;
  writeln('soma = ', s)
end.
```

a saída é:

```
Program Soma
  Declaracoes
    Decl integer
      Var n
      Var i
      Var s
  Corpo
    Readln
      Var n
    Assign i
      Int 1
    Assign s
      Int 0
    While
      <=
        Var i
        Var n
      Bloco
        Assign s
          +
            Var s
            Var i
        Assign i
          +
            Var i
            Int 1
    Writeln
      String 'soma = '
      Var s
```

3.3 Erros

Quando encontra um erro de sintaxe, o Bison chama a função `yyerror`, que indica a linha do erro (`%option yylineno` no Flex). Para o ficheiro `tests/erro.pas`, onde falta o segundo operando de uma soma:

```
Linha 4: erro de sintaxe
```

4 Conclusão

Neste trabalho foi desenvolvido um analisador léxico e sintático para um subconjunto de Pascal, usando o Flex e o Bison. O programa reconhece os tokens da linguagem, verifica a sintaxe, constrói a AST e imprime-a, indicando a linha onde ocorrem erros.

Foi possível pôr em prática a matéria das aulas: expressões regulares e as regras de desambiguação do Flex, gramáticas independentes de contexto, a resolução de ambiguidades com precedências e associatividade, e a construção da AST nas ações semânticas.

O trabalho tem algumas limitações: não faz análise semântica (por exemplo, não verifica se as variáveis foram declaradas ou se os tipos estão certos), as palavras reservadas têm de ser escritas em minúsculas e o ficheiro de entrada tem de ser dado como argumento.

5 Referências

- Slides das aulas teóricas e folhas das aulas práticas da unidade curricular
- Manual do GNU Bison: https://www.gnu.org/software/bison/manual/
- Manual do Flex: https://westes.github.io/flex/manual/
- Documentação do Free Pascal: https://www.freepascal.org/docs.html